Markdown to PDF Generator

This is the standalone CLI version of the `markdown-to-pdf-action`.

> This documentation is also available as a PDF document generated using the abovementioned action. The PDF file can be found under `docs/README.pdf`

Setup

1 Install required python dependencies:

```
markdown
pygments
weasyprint
pypdf
```

> Note if running on MacOS, `pango` is also required: `brew install pango`

2 Download `topdf` script:

```
curl -O https://raw.githubusercontent.com/IsraTech-Software/Markdown-PDF-Generator/refs/heads/main/topdf
chmod +x topdf
```

3 Add the tool to your default environment path.

Features

Base Usage

To generate PDF files from markdown, simply run the script against your markdown files:

```
topdf -i README.md -o output.pdf
```

Branding and styling

With the provided `styles` files you can add a company logo on the top right corner of a document using:

```
topdf -i README.md -o output.pdf -l logo.png
```

To use a custom css file, pass it as an argument:

```
topdf -i README.md -o output.pdf -l logo.png -s styles.css
```

Metadata and Hidden Metadata for AI Screening Tools

The tool now also includes a `-m / --metadata` tag that allows embedding the entirety of the document's text as a string within the rendered pdf for AI-Screening-Tools. The text is placed off screen and never rendered in actual PDF-Viewers, but is very much parseable by AI.

Furthermore, the tool attempts to extract metadata from the text and appends it as normal PDF-Metadata (Title / Description and Author).

```
topdf -i README.md -o output.pdf [-l logo.png] [-s styles.css] [-m]
```

Furthermore, the tool allows the use of the `-j / --job [job_description_file.md]` parameter to further append relevant metadata as keywords. This can be used alongside the `--metadata` parameter:

```
topdf -i README.md -o output.pdf [-l logo.png] [-s styles.css] [-m] [-j
job.md]
```

> Responsible use of the metadata parameters is up to the user.

Demo

CSS demo files along with generated PDF files can be found under `examples/`

> Each example contains a markdown file, alongside a CSS file. All PDF files (aside from the envelope) were generated using `topdf -i example.md -o output.pdf -s styles.css` for each of the respective stylesheets.
>
> Furthermore, some custom CSS classes are used alongside the normal formatting. To check their usage, open the respective `example.md` files and search for the `< ! - - TODO - - >` comments.

The following CSS profiles are currently provided:

Stylesheet	Description	Source and PDF
`built-in`	The default stylesheet, built into the python script	`Markdown`; `PDF`
`Corporate Blue`	Corporate-Style A4 Documentation	`Markdown`; `PDF`
`Formal Letter (A4)`	Formal A4 Letter (German "Anschreiben"), **requires HTML**.	`Markdown`; `PDF`
`Legal Contracts`	German-Style legal contract with § Paragraphs, Absatz (Abs. / Subsection), Satz (S. / Sentences) and Nummer (Nr. / Numbered points)	`Markdown`; `PDF`
`DL Envelope`	(*) DL-Envelope - Please use the `envelope` generation script (See section `Envelope Usage` below)	`PDF`

Evelope Usage

> The envelope tool requires the `topdf` tool to be available in the current environment and wont run without it.

To use the envelope tool:

1 Install the required dependencies:

```
PyMuPDF
opencv-python
numpy
Pillow
```

2 Download the `envelope` script:

```
curl -O https://raw.githubusercontent.com/IsraTech-Software/Markdown-
PDF-Generator/refs/heads/main/examples/envelope-dl/envelope
chmod +x envelope
```

3 Adjust the `envelope` script's properties to match your name and address:

```
# Line 18
LOGO_BASE64: str = "" # TODO your Logo as Base64. If undesired, set
to empty string.
# Lines 20-42
# Details displayed on envelope front (Sender info)
SENDER: Dict[str, str] = {
    'name': 'Your name and last name', # Required
    'address': 'Your address lane 1', # Main address (Required)
    'address2': 'Your address lane 2', # Apartment / Floor (Optional)
    'address3': 'Your address lane 3', # Additional details, if
relevant (Optional)
    'city_and_postcode': 'Your ZIP, City', # Required
    'country': 'Your Country', # (Optional)
    'phone': 'Your phone number' # The script auto appends a "Tel-"
prefix (Optional)
}
# Corporate branding displayed on back of envelope
COMPANY: Dict[str, str] = {
    'name': 'Your company name', # (Optional)
    'address': 'Your company address lane 1', # (Optional)
    'address2': 'Your company address lane 2', # (Optional)
    'address3': 'Your company address lane 3', # (Optional)
    'city_and_postcode': 'Your company\'s ZIP, City', # (Optional)
    'country': 'Your Company\'s country', # (Optional)
    'phone': 'Your Company\'s phone number', # (Optional)
    'website': 'https://isratech.software/', # (Optional)
    'email': 'sales@isratech.software' # (Optional)
}
```

4 Generate your first envelope:

```
envelope
```

> The script first attempts to capture a multi-lane address and auto-maps each newline to an address field (Great for copy-pasting addresses). To continue / manually ender the address, hit `CTRL+D`.

Embedding a stamp (Detusche Post)

To embed a stamp, run the script against the stamp file:

```
envelope Briefmarke.pdf
```

An example stamp PDF file can be found here.

> The script only works if theres exactly one stamp in the PDF document, as it tracks the borders around the stamp and uses them to crop the image. If multiple stamps are present, the first one is always extracted.